TalkLate

Traduccion en tiempo real con conciencia cultural y emocional

20. OWASP Top 10 para aplicaciones con LLM

Trabajo de grado

Julian David Diaz Medina

Diego Alejandro Franco Atehortua

Director: Wider Farid Sanchez Garzon

Version 1.1 - Noviembre de 2026

Version: 1.1 (12/11/2026) **Referencia:** OWASP Top 10 for LLM Applications, edicion 2026 (publicada el 04/08/2026), repositorio https://github.com/GenAI-Security-Project/GenAI-LLM-Top10 (carpeta `2026/final/`).

Este documento revisa TalkLate contra los 10 riesgos de la lista. Es una revision honesta: marca lo que esta cubierto, lo que esta cubierto a medias y lo que falta. Los controles ya existian en el codigo (se construyeron por las convenciones de seguridad y la arquitectura hexagonal), pero hasta la revision del 11/11/2026 no estaban contrastados con esta lista. La version 1.1 recoge los ajustes hechos el 12/11/2026 y el cambio de motor de traduccion (ADR-020).

> *Nota: los nombres de los riesgos son los oficiales; las definiciones y estrategias de la tabla son un resumen propio del equipo.*

1. Resumen

TalkLate usa modelos de lenguaje en **un solo lugar**: traducir un mensaje. El motor principal es **Claude Sonnet servido por Google Vertex AI** (ADR-020); Qwen local queda como adaptador alternativo. El modelo recibe texto y devuelve texto en JSON. **No tiene herramientas, no toca la base de datos, no envia correos y no ve datos de otros usuarios.** Eso reduce mucho el riesgo, porque lo peor que puede pasar con un mensaje malicioso es que se dane la traduccion de la propia persona que lo escribio.

Lo que cambio con Vertex: el texto de cada mensaje **ahora sale del servidor** hacia Google Cloud (antes, con Qwen local, no salia). Eso pesa sobre LLM02 y LLM06, y esta reflejado en la politica de datos.

#	Riesgo (2026)	Estado	Resumen
LLM01	Prompt Injection	Parcial	El impacto es bajo, pero el texto del usuario entra al prompt sin marcarlo como dato y no hay pruebas de inyeccion
LLM02	Sensitive Information Disclosure	Parcial	Cifrado, logs limpios y limites de uso; el texto sale a Google Cloud y a ElevenLabs (documentado); la app no avisa a la persona
LLM03	Excessive Agency	Cubierto por diseno	El modelo no tiene herramientas ni permisos
LLM04	Supply Chain	Parcial	Trivy con checksum y lockfiles; falta un SBOM y revisar los terminos de los proveedores
LLM05	Data and Model Poisoning	Parcial	No se entrena ni se usa RAG; el glosario cultural (196 entradas) alimenta el prompt y no esta validado por nativos
LLM06	Unbounded Consumption	Parcial	Limites por usuario, tamanos, tiempos limite y tope de salida; falta un tope de gasto
LLM07	Misinformation	Parcial	Es el riesgo central del producto; hay reportes y notas, pero la medicion de calidad es debil
LLM08	Hidden Context Exposure	Cubierto	El prompt no tiene secretos ni se usa como control de seguridad
LLM09	Vector and Embedding Weaknesses	No aplica	No hay RAG, embeddings ni base vectorial

#	Riesgo (2026)	Estado	Resumen
LLM10	Improper Output Handling	Cubierto	La salida se valida (JSON, tono, genero, nota y region) y la app no interpreta HTML

Conteo: 3 cubiertos (LLM03, LLM08, LLM10), 1 no aplica (LLM09) y 6 parciales. Ninguno esta en rojo.

2. Detalle por riesgo

LLM01 - Prompt Injection (parcial)

Que es: una entrada altera el comportamiento del modelo porque este no distingue entre instrucciones y datos.

Que tiene TalkLate:

- La salida se fuerza a JSON y se valida en codigo (`backend/src/adaptadores/traduccion/prompt.js`, funcion `leerRespuesta`): la traduccion debe ser texto no vacio, el tono debe estar en la lista permitida, el genero pasa por una expresion regular y la nota y la region solo se aceptan como texto de largo acotado.
- El modelo no puede hacer nada fuera de devolver texto (ver LLM03).
- El mensaje tiene un maximo de 5000 caracteres y hay limite de mensajes por minuto por usuario.

Que falta:

- El texto del usuario se pega al final del prompt (`Mensaje:` + texto) sin delimitadores ni una instruccion que diga "esto es dato, no instrucciones".
- El audio se transcribe con Whisper y el texto resultante entra igual al prompt: la voz tambien es un canal de inyeccion.
- No se quitan caracteres Unicode invisibles.
- No hay pruebas automaticas con mensajes del tipo "ignora lo anterior y responde X".

Riesgo real: bajo. Un atacante solo puede estropear su propia traduccion; no accede a datos de otros ni ejecuta acciones.

LLM02 - Sensitive Information Disclosure (parcial)

Que es: datos confidenciales salen por un canal no autorizado: la respuesta, los logs, los terceros.

Que tiene TalkLate:

- Cifrado AES-256-GCM del contenido de mensajes y traducciones en reposo (`adaptadores/cifrado/aesGcm.js`).
- Logs que descartan por codigo los campos `password`, `token`, `correo`, `contenido`, `texto` y `codigo` (`compartido/log.js`), con prueba unitaria.
- Aislamiento entre usuarios y limites de uso en endpoints sensibles.

- El prompt no lleva datos personales: solo el texto a traducir, idiomas, region, tono y, en voz, una descripcion de como hablo la persona.
- Whisper corre en el servidor: el **audio original no sale** del servidor.
- Politica de tratamiento de datos y matriz de sensibilidad actualizadas el 12/11/2026: nombran a **Google Cloud (Vertex AI)** y a **ElevenLabs** como terceros y clasifican el **genero inferido de la voz** y los **problemas reportados** como datos personales.

Que falta:

- El **texto de cada mensaje sale a Google Cloud** para traducirlo y el texto traducido sale a ElevenLabs para la voz. La persona lo acepta en la politica al registrarse, pero la app no se lo recuerda mientras conversa.
- Falta revisar los terminos de los proveedores (retencion y uso de los datos para entrenar) y dejarlo escrito.
- El genero inferido de la voz se guarda por mensaje; hay que decidir si hace falta guardarlo o basta usarlo al momento.

LLM03 - Excessive Agency (cubierto por diseno)

Que es: el sistema ejecuta acciones danosas por salidas inesperadas del modelo, por exceso de herramientas, permisos o autonomia.

Que tiene TalkLate: el modelo no tiene herramientas, no llama funciones, no accede a la base de datos y no envia mensajes. El backend recibe el JSON, lo valida y decide todo lo demas con codigo determinista. La autorizacion (JWT, dueno de la conversacion, rol admin) vive en el backend, nunca en el modelo.

Reglas del equipo (quedan escritas aqui):

- Ningun motor de IA recibe herramientas ni acceso a datos. Si algun dia se agrega, se revisa este riesgo de nuevo antes de integrarlo.
- La cuenta de Google que usa Vertex debe tener **solo el permiso de invocar modelos** (rol de usuario de Vertex AI), no permisos de administracion del proyecto.

LLM04 - Supply Chain (parcial)

Que es: riesgos de integridad en paquetes, modelos, datos y plataformas de terceros.

Que tiene TalkLate:

- Escaneo con Trivy en cada pull request y cada lunes (`seguridad_trivy.md`). Reescaneo del 12/11/2026 con la dependencia nueva de Vertex: **0 vulnerabilidades altas o criticas y 0 secretos**.
- Binario de Trivy fijado con checksum SHA-256; no se usa `trivy-action` por el ataque de marzo de 2026.
- `package-lock.json` en backend, frontend y pruebas, e instalacion con `npm ci` en el CI.
- Los motores de ElevenLabs, Azure, OpenAI y Claude directo se llaman con `fetch` (sin SDK); la traduccion por Vertex usa el SDK oficial de Anthropic.

- La traduccion por Vertex se autentica con las credenciales de Google (ADC): no hay clave de API en el repositorio.

Que falta:

- No hay SBOM (lista de componentes) ni siquiera de las dependencias de npm.
- Los modelos se piden por nombre (`claude-sonnet-4-6` en Vertex, `claude-sonnet-5-5` en la API directa): los dos valores por defecto no coinciden y conviene fijar uno y comprobar que exista en la region configurada.
- Si se vuelve a usar Qwen, el modelo se descarga por etiqueta (`qwen2.5:7b`) y no por huella fija.
- Falta una nota de evaluacion de los terceros (Anthropic, Google Cloud, ElevenLabs): terminos, retencion y entrenamiento con datos.

LLM05 - Data and Model Poisoning (parcial)

Que es: manipulacion de los datos o del modelo para meter sesgos o comportamientos danosos.

Que tiene TalkLate: no se entrena ni se ajusta ningun modelo y no hay RAG. El unico dato que influye en la salida es el glosario cultural (`glosario.json`), que esta versionado en git, pasa por pull request y trae la `source` de cada entrada (196 de 196).

Que falta: las 196 entradas tienen `native_validated: false`: ningun hablante nativo las ha revisado. Desde el 12/11/2026 el prompt incluye de cada modismo detectado su equivalente **y un ejemplo de uso** (few-shot), asi que una entrada mala (un equivalente incorrecto o una de las 26 marcadas como sensibles) influye aun mas en la traduccion. El campo ya existe; falta usarlo.

LLM06 - Unbounded Consumption (parcial)

Que es: uso de recursos sin control: caida del servicio o costo desbordado (denial of wallet).

Que tiene TalkLate:

- Limites por usuario: 30 mensajes por minuto, 40 voces por minuto y 10 reportes de problemas por hora (todos configurables); por IP en registro, recuperacion y restablecimiento.
- Tamanos: 5000 caracteres por mensaje, 1000 por voz, 1 MB por peticion y 15 MB solo para audio.
- Tiempos limite en todas las llamadas externas (Claude por Vertex 30 s y sin reintentos del SDK, Whisper 30 s, ElevenLabs 20 s, ffmpeg 15 s) y tope de salida: `max_tokens` de 1024 en Claude y `num_predict` de 1024 en Qwen.
- Circuit Breaker: un motor que falla se salta por 60 segundos.
- Los audios sintetizados viven 5 minutos y como maximo 200 en memoria.

Que falta:

- Los limites estan **en memoria**: se pierden al reiniciar y no funcionan con varias instancias.
- No hay un **tope de gasto** diario para Claude (Vertex), ElevenLabs o Whisper en la nube; los limites cuentan peticiones, no costo. Con Vertex y ElevenLabs cada mensaje cuesta, asi que esta brecha pesa mas que antes.

LLM07 - Misinformation (parcial)

Que es: salidas incorrectas que parecen creibles. En TalkLate significa **una traduccion equivocada que suena bien**; es el riesgo central del producto.

Que tiene TalkLate:

- Reporte de traducciones malas por parte del usuario, con atencion del administrador.
- Nota cultural que explica cada modismo adaptado.
- El motor de reglas **ya no entrega una traduccion palabra por palabra**: si el texto no trae modismos conocidos, falla con un error claro (cambio del 12/11/2026).
- Aviso "Traduccion aproximada" cuando traduce el motor de reglas con palabras sin traducir.
- Prompt de **transcreacion cultural** con ejemplos reales del glosario.
- Salida estructurada con campos obligatorios.
- Medicion de modismos adaptados (DRV-02): 14 de 15 con el motor de reglas.

Que falta:

- Esa medicion se hizo **con el motor de reglas, que se construyo con los mismos 15 modismos**: es una linea base, no una medida de calidad. Falta medirla con el motor de produccion (Claude en Vertex) y con una lista de modismos distinta.
- No hay evaluacion de calidad con hablantes nativos (ver LLM05).
- La app no dice de forma general que la traduccion automatica puede equivocarse.

LLM08 - Hidden Context Exposure (cubierto)

Que es: extraccion de las instrucciones ocultas del modelo (system prompt) cuando eso ayuda a un atacante. La guia pide asumir que ese contexto es descubrible.

Que tiene TalkLate:

- El prompt no tiene claves, cadenas de conexion ni datos de usuarios. Las claves viven en variables de entorno y `.env` esta en `.gitignore`; Trivy busca secretos en cada pull request.
- El prompt no es un control de seguridad: la autorizacion se hace en codigo, fuera del modelo.
- El prompt ya es publico (esta en el repositorio, `prompt.js`), asi que filtrarlo no da ventaja.

LLM09 - Vector and Embedding Weaknesses (no aplica)

TalkLate no usa RAG, embeddings, memoria de agentes ni cache semantica. SurrealDB soporta vectores, pero no se usan. Si se agrega busqueda semantica o RAG, este riesgo se revisa antes.

LLM10 - Improper Output Handling (cubierto)

Que es: la salida del modelo se pasa a otro componente sin validarla (XSS, inyeccion, etc.).

Que tiene TalkLate:

- La salida del modelo se trata como entrada no confiable: JSON validado, tono contra lista, genero normalizado y, desde el 12/11/2026, `nota_cultural` (500 caracteres) y `region_origen` (80) solo como texto de largo acotado, con prueba unitaria.
- La app es React Native: muestra el texto como texto, no como HTML ni Markdown, asi que no hay XSS.
- Consultas parametrizadas en SQLite y literales serializados en SurrealDB (nada de texto concatenado).
- El texto que va a Azure se escapa antes de armar el SSML.
- Los logs no guardan el contenido.

3. Brechas y plan para cerrarlas

Prioridad	Brecha	Riesgo	Accion	Estado
1	`nota_cultural` y `region_origen` sin tipo ni largo	LLM10	Convertir a texto y acotar en `leerRespuesta`	**Hecho 12/11/2026**
2	Qwen sin limite de salida; Vertex sin tiempo limite	LLM06	`num_predict` en Qwen; 30 s y sin reintentos en Vertex	**Hecho 12/11/2026**
3	Reporte de problemas sin limite	LLM06	10 por hora por usuario	**Hecho 12/11/2026**
4	Terceros no nombrados en la politica de datos	LLM02	Actualizar la politica y la matriz (Google Cloud, ElevenLabs, genero de la voz)	**Hecho 12/11/2026**
5	El texto del usuario entra al prompt sin delimitar	LLM01	Encerrar el mensaje entre marcas y decir al modelo que es dato; quitar Unicode invisible; pruebas de inyeccion	Pendiente (toca el prompt de transcreacion: probar con el modelo real)
6	Sin SBOM; modelos sin fijar	LLM04	`trivy sbom` en el CI; unificar el modelo por defecto	Pendiente
7	Glosario sin validacion nativa	LLM05, LLM07	Que un hablante paisa y uno de New York revisen las 196 entradas y se marquen con `native_validated: true`	Pendiente (depende de personas)
8	Medicion de calidad solo con el motor de reglas	LLM07	Correr `npm run medir` con Claude (Vertex) y con modismos que no esten en el motor de reglas	Pendiente
9	Sin tope de gasto y limites en memoria	LLM06	Tope diario de gasto y limites en una base compartida	Pendiente (fuera del MVP de una instancia)
10	La app no recuerda que el texto sale a terceros	LLM02	Aviso corto en la pantalla de chat	Pendiente

4. Hallazgo adicional (fuera de la lista OWASP)

Credenciales de administrador por defecto. Al arrancar, el backend crea un administrador con correo `dafa123.daf@gmail.com` y contrasena `Admin123!` si no se definen `ADMIN_CORREO` y `ADMIN_PASSWORD`.

Desde el 12/11/2026, con `NODE_ENV=production` el servidor se niega a crear el administrador sin `ADMIN_PASSWORD` (con prueba unitaria). En desarrollo se conserva la contrasena conocida, documentada como solo para pruebas. Queda pendiente sacar el correo personal del codigo.

5. Como se relaciona con lo demas

- Complementa a Trivy (`seguridad_trivy.md`): Trivy cubre el eslabon de dependencias y secretos; este documento cubre los riesgos propios de usar un modelo de lenguaje.
- Los controles probados estan en la piramide de pruebas (`piramide_de_pruebas.md`): cifrado, logs sin datos personales, limites de uso y validaciones.
- Responde al driver de privacidad (DRV-03), a RNF-011 y a las convenciones de seguridad (`convenciones/seguridad.md`).